

Password Strength Checker

DecodeLabs Cyber Security Internship

Project Report

Submitted By

Name: Muhammad Sameer

Organization: DecodeLabs

Project 1: Password Strength Checker

1. Introduction

Passwords are the first line of defense in protecting digital accounts and sensitive information. Weak passwords increase the risk of unauthorized access, brute-force attacks, and credential theft.

This project implements a Password Strength Checker using Python. The application evaluates a password based on its length, uppercase letters, numeric digits, and special characters, then classifies it as Weak, Medium, or Strong.

2. Objective

The main objective of this project is to:

- Understand password validation techniques.
- Apply Python programming concepts.
- Learn basic cybersecurity principles.
- Evaluate password strength based on predefined security rules.

3. Project Requirements

The program checks:

- Password length
- Uppercase letters
- Numbers
- Special characters
- Password strength level

4. Technologies Used

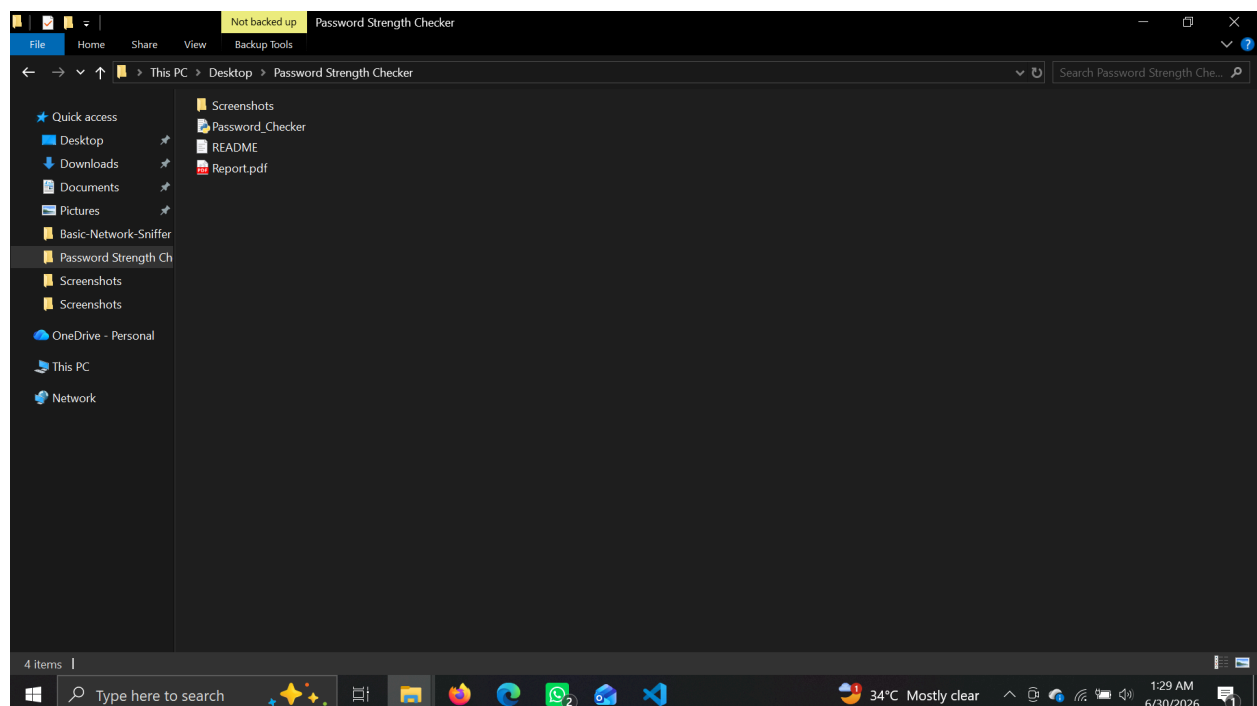
- Python 3
- Visual Studio Code
- Windows Operating System

5. Methodology

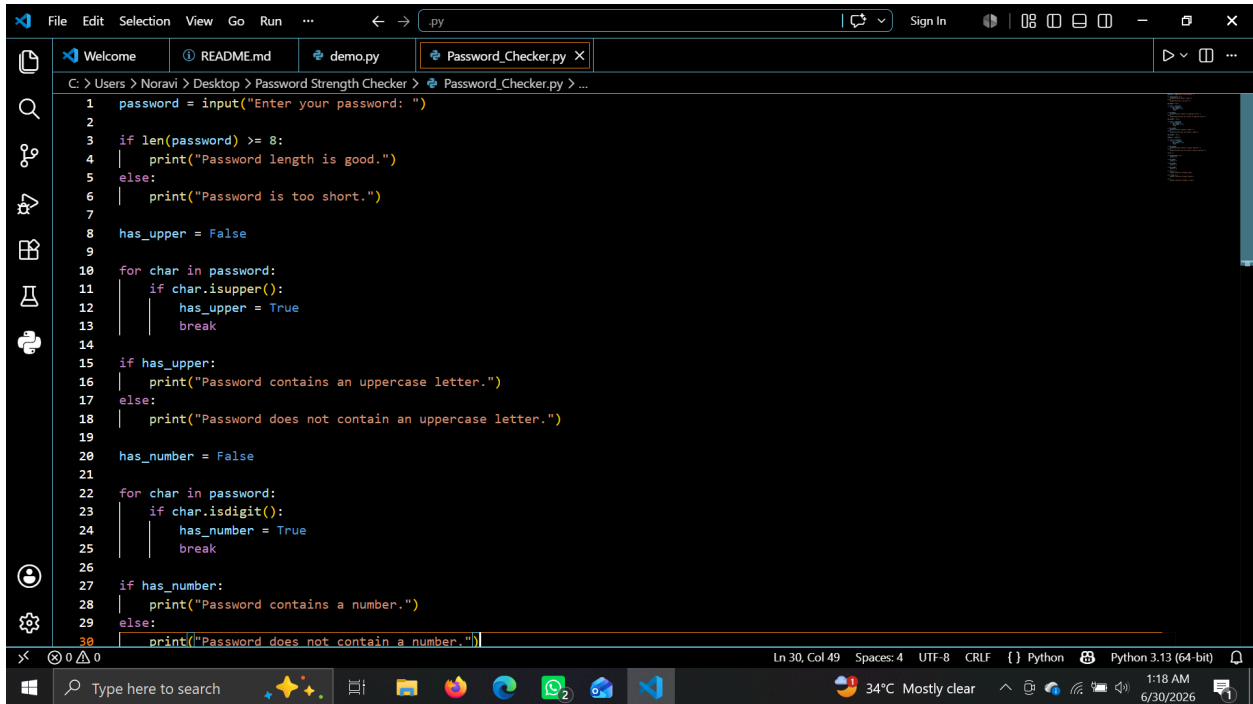
The program performs the following steps:

1. Accepts password input from the user.
2. Checks whether the password length is at least 8 characters.
3. Verifies the presence of uppercase letters.
4. Detects numeric digits.
5. Detects special characters.
6. Calculates a strength score.
7. Displays the final password strength.

Project Folder



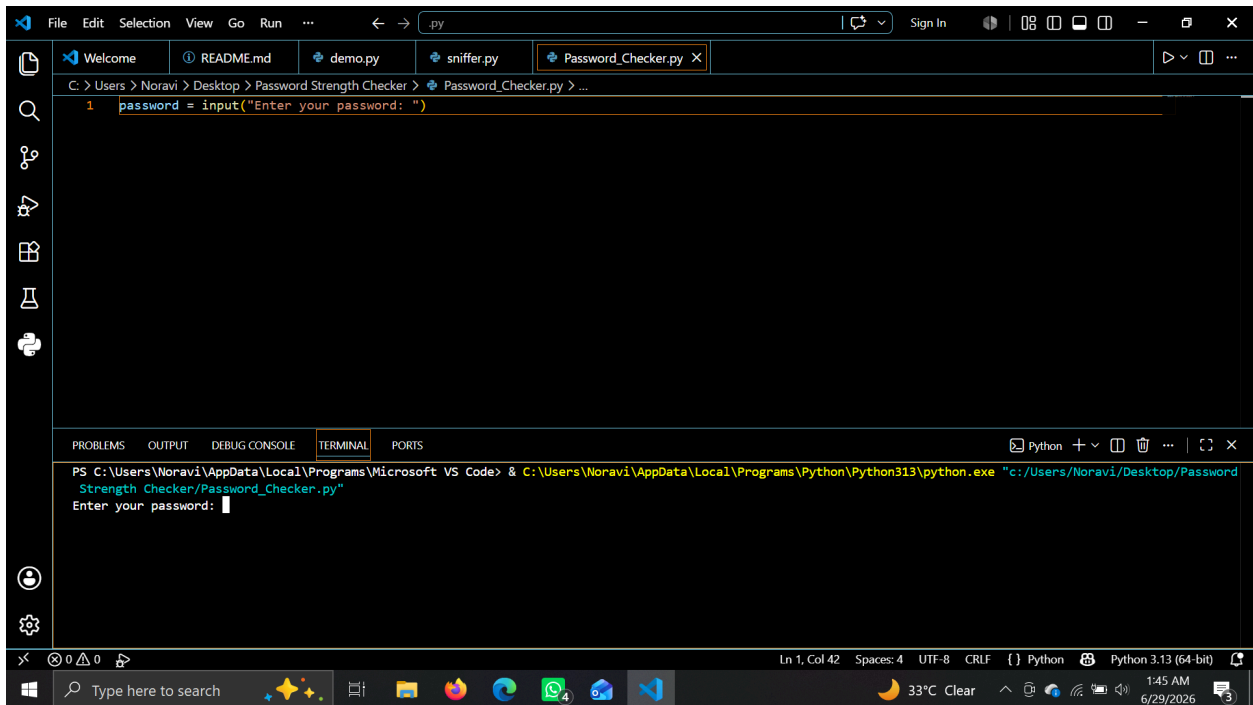
Python File (Code)



```
1 password = input("Enter your password: ")
2
3 if len(password) >= 8:
4     print("Password length is good.")
5 else:
6     print("Password is too short.")
7
8 has_upper = False
9
10 for char in password:
11     if char.isupper():
12         has_upper = True
13         break
14
15 if has_upper:
16     print("Password contains an uppercase letter.")
17 else:
18     print("Password does not contain an uppercase letter.")
19
20 has_number = False
21
22 for char in password:
23     if char.isdigit():
24         has_number = True
25         break
26
27 if has_number:
28     print("Password contains a number.")
29 else:
30     print("Password does not contain a number.")
```

6. Implementation

Step 1 – User Input



```
1 password = input("Enter your password: ")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: 
```

Step 2 – Length Check

The image shows a Windows desktop environment. In the foreground, a Visual Studio Code (VS Code) editor window is open. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', and a search icon. The left sidebar contains icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, and Cloud. The main editor area shows a file named 'Password_Checker.py' with the following Python code:

```
1 password = input("Enter your password: ")
2
3 if len(password) >= 8:
4     print("Password length is good.")
5 else:
6     print("Password is too short.")
```

The bottom of the VS Code window features a panel with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, displaying the command prompt output:

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: abc
Password is too short.
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> |
```

The Windows taskbar at the bottom shows the Start button, a search bar with the text 'Type here to search', and several pinned application icons including File Explorer, Firefox, Chrome, VS Code, and a chat application. The system tray on the right indicates the time as 1:55 AM on 6/29/2026.

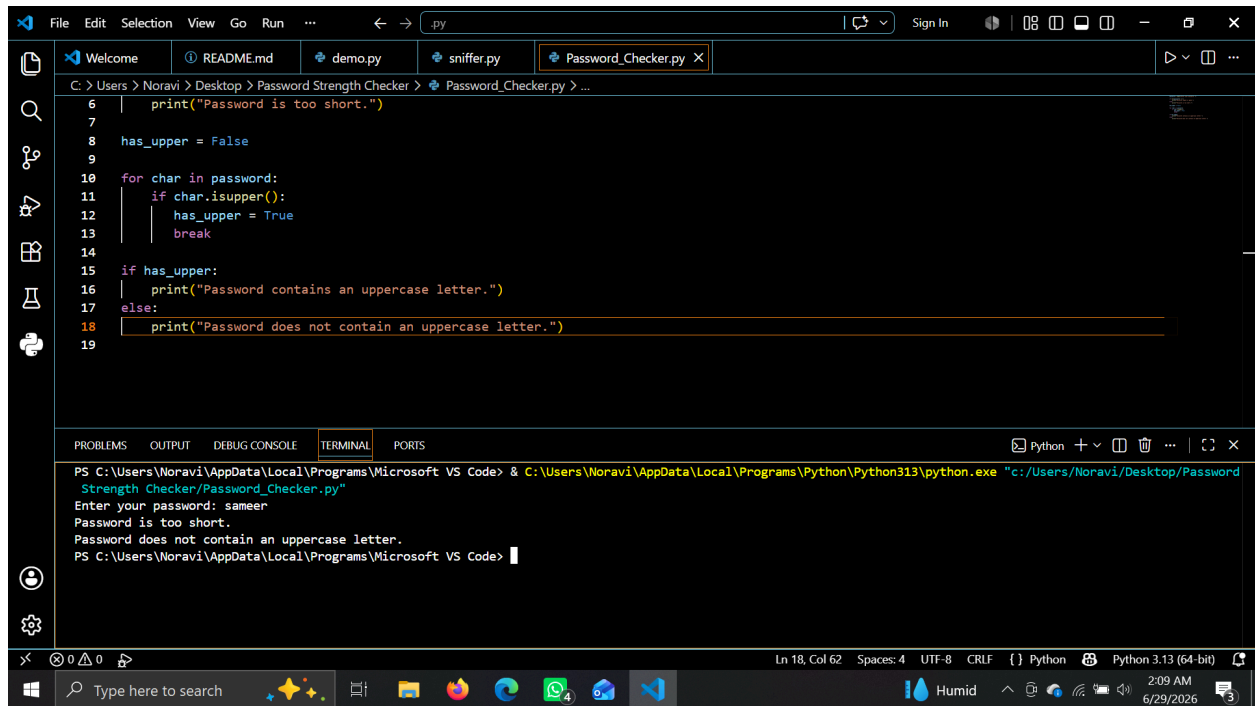
The screenshot displays a Windows 10 desktop environment. The primary focus is the Visual Studio Code (VS Code) application, which is open with a Python file named `Password_Checker.py`. The code in the editor is as follows:

```
1 password = input("Enter your password: ")
2
3 if len(password) >= 8:
4     print("Password length is good.")
5 else:
6     print("Password is too short.")
```

Below the code editor, the **TERMINAL** panel is active, showing the execution of the script. The command prompt indicates the current directory is `C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe` and the script is being run from `c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py`. The output shows that the user entered `password1122`, which is correctly identified as having a good password length.

The Windows taskbar at the bottom shows the Start button, a search bar, and several pinned applications including File Explorer, Firefox, Chrome, and VS Code. The system tray on the right indicates the date and time as 6/29/2026, 1:56 AM.

Step 3 – Uppercase Check

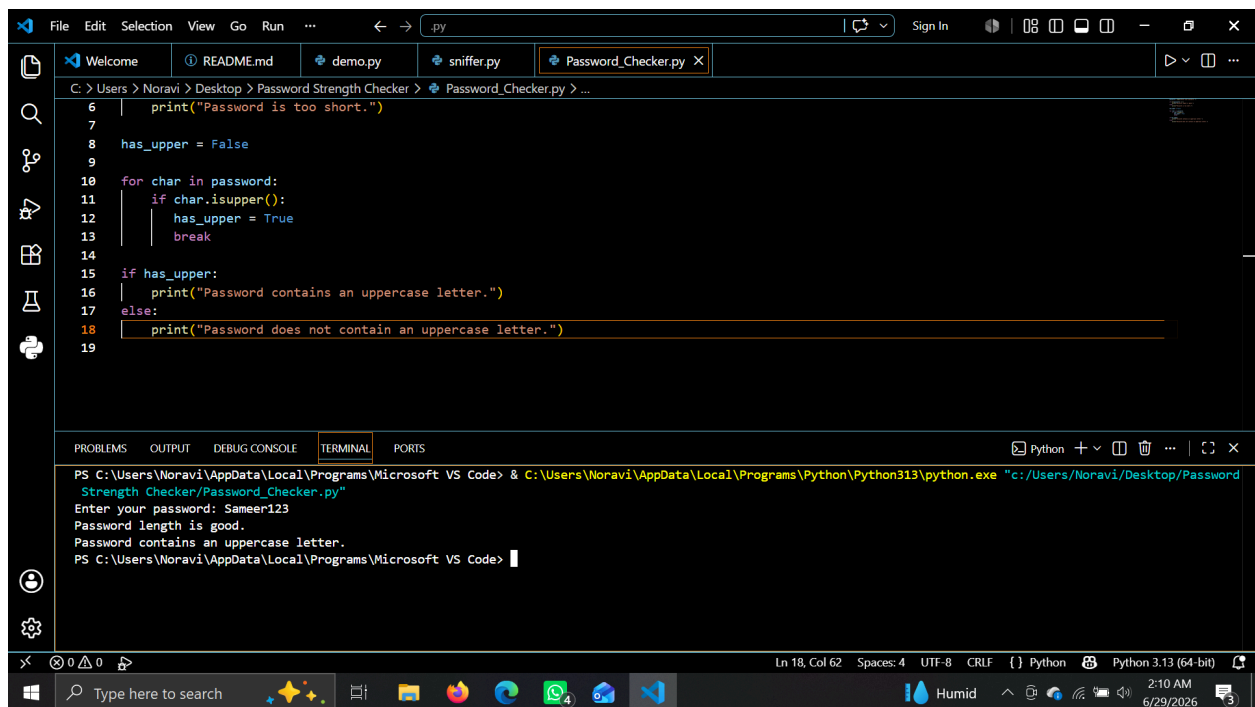


The screenshot shows the Visual Studio Code editor with the file `Password_Checker.py` open. The code implements an uppercase check for a password. The terminal output shows the program running and checking the password "sameer".

```
6 | print("Password is too short.")
7 |
8 | has_upper = False
9 |
10 | for char in password:
11 |     if char.isupper():
12 |         has_upper = True
13 |         break
14 |
15 | if has_upper:
16 |     print("Password contains an uppercase letter.")
17 | else:
18 |     print("Password does not contain an uppercase letter.")
19 |
```

Terminal Output:

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: sameer
Password is too short.
Password does not contain an uppercase letter.
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>
```



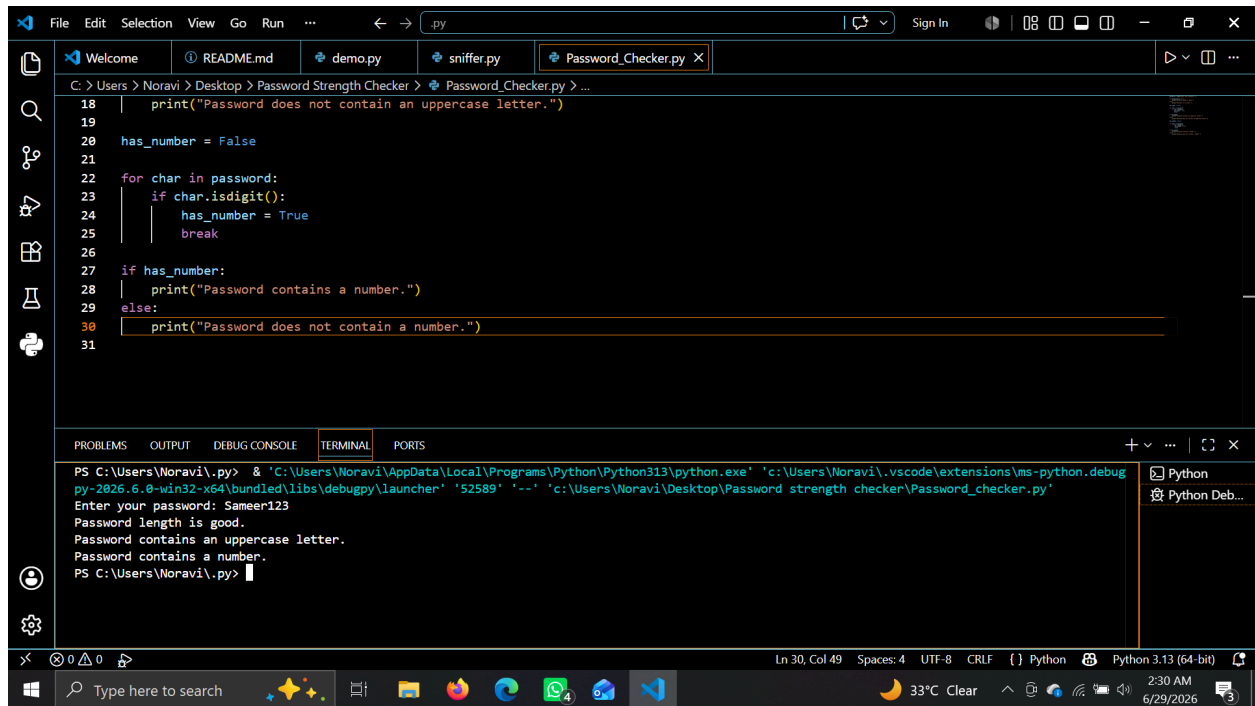
The screenshot shows the Visual Studio Code editor with the file `Password_Checker.py` open. The code implements an uppercase check for a password. The terminal output shows the program running and checking the password "Sameer123".

```
6 | print("Password is too short.")
7 |
8 | has_upper = False
9 |
10 | for char in password:
11 |     if char.isupper():
12 |         has_upper = True
13 |         break
14 |
15 | if has_upper:
16 |     print("Password contains an uppercase letter.")
17 | else:
18 |     print("Password does not contain an uppercase letter.")
19 |
```

Terminal Output:

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: Sameer123
Password length is good.
Password contains an uppercase letter.
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>
```

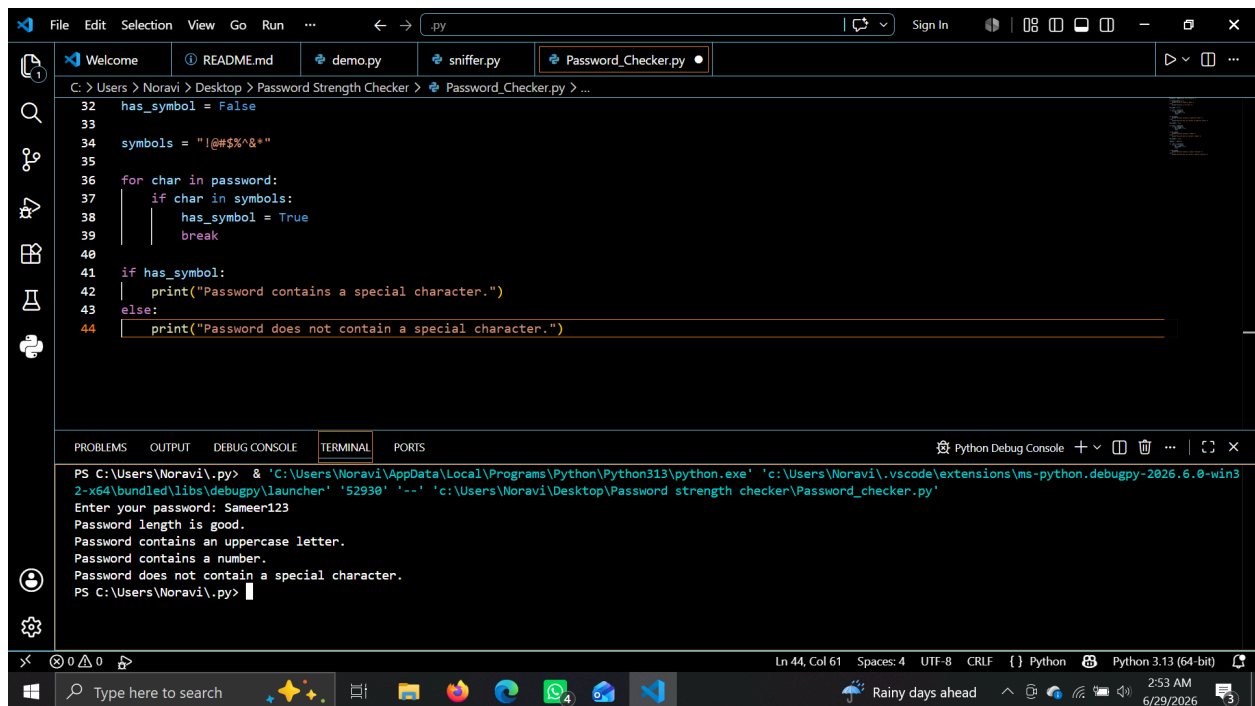
Step 4 – Number Check



```
File Edit Selection View Go Run ... .py
Welcome README.md demo.py sniffer.py Password_Checker.py X
C:\Users\Noravi\Desktop>Password Strength Checker> Password_Checker.py > ...
18 | print("Password does not contain an uppercase letter.")
19 |
20 | has_number = False
21 |
22 | for char in password:
23 |     if char.isdigit():
24 |         has_number = True
25 |         break
26 |
27 | if has_number:
28 |     print("Password contains a number.")
29 | else:
30 |     print("Password does not contain a number.")
31 |

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Noravi\.py> & 'C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Noravi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52589' '--' 'c:\Users\Noravi\Desktop>Password strength checker>Password_checker.py'
Enter your password: Sameer123
Password length is good.
Password contains an uppercase letter.
Password contains a number.
PS C:\Users\Noravi\.py>
```

Step 5 – Symbol Check



```
File Edit Selection View Go Run ... .py
Welcome README.md demo.py sniffer.py Password_Checker.py
C:\Users\Noravi\Desktop>Password Strength Checker> Password_Checker.py > ...
32 | has_symbol = False
33 |
34 | symbols = "!@#$%^&*"
35 |
36 | for char in password:
37 |     if char in symbols:
38 |         has_symbol = True
39 |         break
40 |
41 | if has_symbol:
42 |     print("Password contains a special character.")
43 | else:
44 |     print("Password does not contain a special character.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Noravi\.py> & 'C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Noravi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52930' '--' 'c:\Users\Noravi\Desktop>Password strength checker>Password_checker.py'
Enter your password: Sameer123
Password length is good.
Password contains an uppercase letter.
Password contains a number.
Password does not contain a special character.
PS C:\Users\Noravi\.py>
```

The screenshot shows the Visual Studio Code editor with the file `Password_Checker.py` open. The code checks for the presence of numbers and symbols in a password. The terminal output shows the execution of the script with the password `Sameer@123`, which is deemed to have good length and contains uppercase letters, numbers, and special characters.

```
27 if has_number:
28     print("Password contains a number.")
29 else:
30     print("Password does not contain a number.")
31
32 has_symbol = False
33
34 symbols = "!@#$%^&*"
35
36 for char in password:
37     if char in symbols:
38         has_symbol = True
39         break
40
41 if has_symbol:
42     print("Password contains a special character.")
43 else:
44     print("Password does not contain a special character.")
```

Terminal Output:

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: Sameer@123
Password length is good.
Password contains an uppercase letter.
Password contains a number.
Password contains a special character.
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>
```

Final password strength

Weak Password

The screenshot shows the Visual Studio Code editor with the file `Password_Checker.py` open. The code checks for password length, uppercase letters, numbers, and symbols, and assigns a strength score. The terminal output shows the execution of the script with the password `Sameer`, which is deemed too short and lacks numbers and symbols, resulting in a 'Weak' password strength.

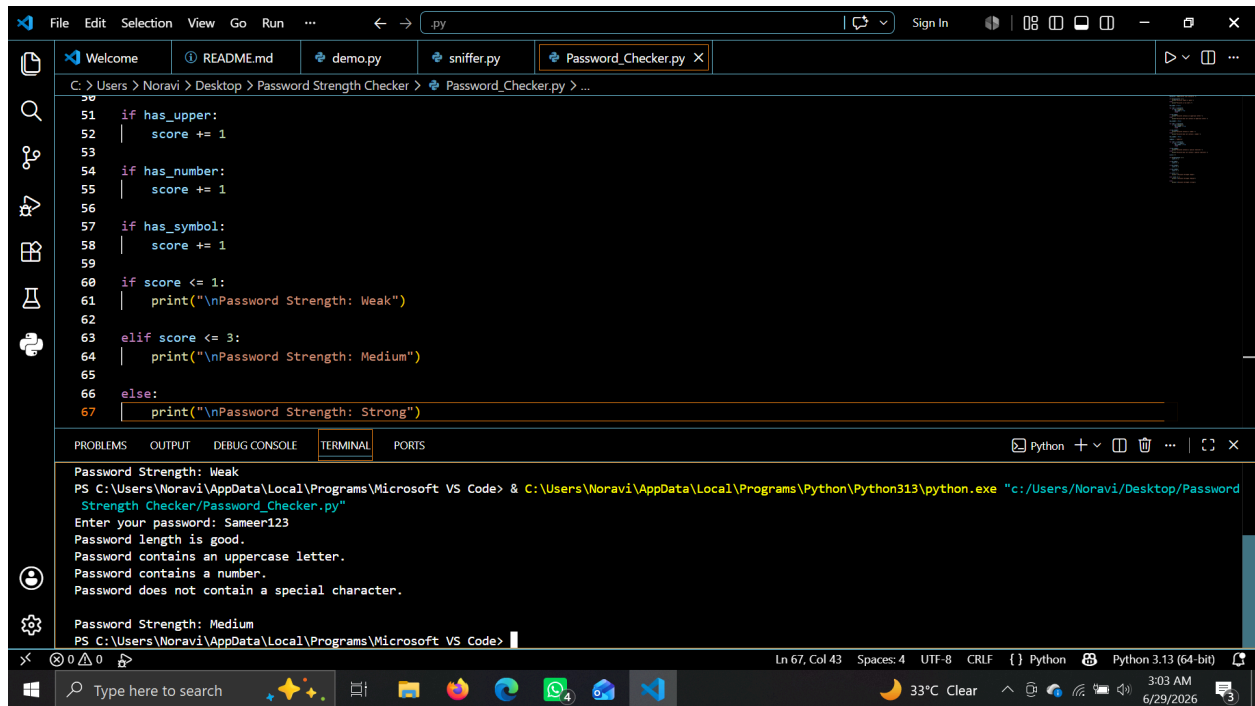
```
51 if has_upper:
52     score += 1
53
54 if has_number:
55     score += 1
56
57 if has_symbol:
58     score += 1
59
60 if score <= 1:
61     print("\nPassword Strength: Weak")
62
63 elif score <= 3:
64     print("\nPassword Strength: Medium")
65
66 else:
67     print("\nPassword Strength: Strong")
```

Terminal Output:

```
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: Sameer
Password is too short.
Password contains an uppercase letter.
Password does not contain a number.
Password does not contain a special character.

Password Strength: Weak
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>
```


Medium Password



The screenshot shows a Visual Studio Code editor with a Python file named `Password_Checker.py`. The script checks password strength based on length, uppercase letters, numbers, and symbols. The terminal output shows the password `Sameer123` is classified as 'Medium'.

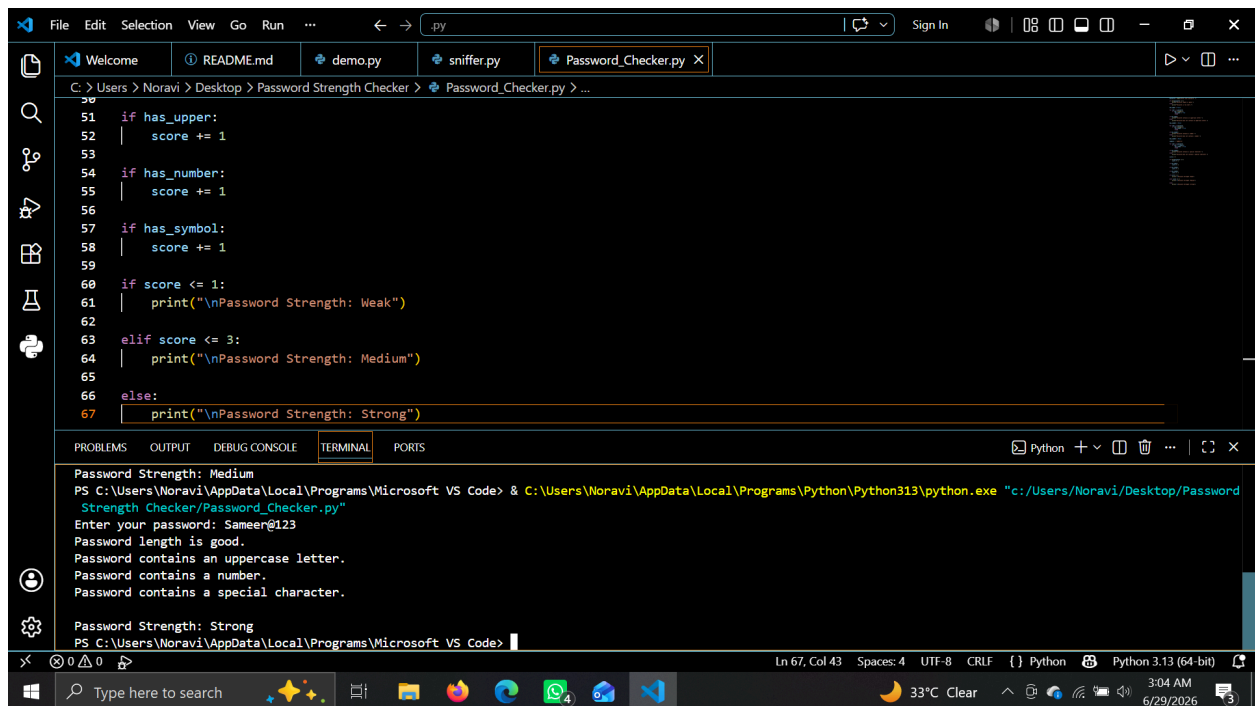
```
50
51 if has_upper:
52     score += 1
53
54 if has_number:
55     score += 1
56
57 if has_symbol:
58     score += 1
59
60 if score <= 1:
61     print("\nPassword Strength: Weak")
62
63 elif score <= 3:
64     print("\nPassword Strength: Medium")
65
66 else:
67     print("\nPassword Strength: Strong")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Password Strength: Weak
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: Sameer123
Password length is good.
Password contains an uppercase letter.
Password contains a number.
Password does not contain a special character.

Password Strength: Medium
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>

Strong Password



The screenshot shows the same Visual Studio Code editor with the `Password_Checker.py` script. The terminal output shows the password `Sameer@123` is classified as 'Strong'.

```
50
51 if has_upper:
52     score += 1
53
54 if has_number:
55     score += 1
56
57 if has_symbol:
58     score += 1
59
60 if score <= 1:
61     print("\nPassword Strength: Weak")
62
63 elif score <= 3:
64     print("\nPassword Strength: Medium")
65
66 else:
67     print("\nPassword Strength: Strong")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Password Strength: Medium
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Noravi\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Noravi/Desktop/Password Strength Checker/Password_Checker.py"
Enter your password: Sameer@123
Password length is good.
Password contains an uppercase letter.
Password contains a number.
Password contains a special character.

Password Strength: Strong
PS C:\Users\Noravi\AppData\Local\Programs\Microsoft VS Code>

7. Features

- User-friendly input
- Length validation
- Uppercase detection
- Number detection
- Special character detection
- Password strength classification

8. Challenges Faced

During development, a few issues were encountered:

- Python indentation errors
- Variable scope (`has_symbol`) issue
- Understanding loops and conditional statements
- Debugging program execution

These challenges were resolved through testing and correcting the program logic.

9. Learning Outcomes

Through this project, I gained practical knowledge of:

- Python programming
- String handling
- Loops
- Conditional statements
- Password validation
- Basic cybersecurity concepts
- Debugging techniques

10. Future Enhancements

Future improvements may include:

- Password entropy calculation
- Common password detection
- Random password generator
- Graphical User Interface (GUI)
- Password breach database integration

11. Conclusion

The Password Strength Checker successfully evaluates passwords using multiple security criteria and classifies them as Weak, Medium, or Strong. This project strengthened my Python programming skills and enhanced my understanding of password security and cybersecurity fundamentals.

